1.

HTML

Ett HTML-dokument består av flertal nästlade HTML-element som i sin tur består av en tagg och ev. attributer och innehåll. Taggen är själva grunden till elementet och beskriver vad för typ av element det är. Exempel på taggar är , <a>, <body>, <img>, m.fl. Ett HTML-element kan även innehålla en eller flera attributer, dessa är egenskaper som anpassar elementet på olika sätt. Olika element kan ha olika typer av attribut, men vissa attribut är globala och går att sätta på (i princip) alla element. Några exempel på attribut är src, height, for, type, class, id. Varav de två sista är globala. Dessa attribut har oftast även ett värde som sätts med =”värde” som styr elementets egenskaper. Innehållet i elementet är det som (mestadels) syns på webbsidan. Det är oftast text eller ett annat element, detta kallas att nästla. Att nestla HTML-element innebär att andra element ligger inom innehållet på ett element, detta är grunden till hur ett HTML-dokument är strukturerat.

CSS

CSS används för att ändra utseendet på HTML-element. Det består till början av selektorer som pekar på vilket element den ska ändra via ex. taggar (div, a, p) eller klasser och id (.class, #id). Därefter deklarerar man vilka egenskaper som man vill ändra, ex. color, width, border och sätter ett värde som är relaterat till egenskapen, ex. en färg till color eller en längd till width.

Box Model

Det är en term som syftar på hur varje HTML-element består av en box (behållare) med 4 olika lager. Det första och innersta lagret är content (innehållet) där text eller andra element ligger. Det andra är padding vilket är en utfyllnad (tomrum) mellan innehållet och nästa lager som är border dvs. kanten på elementet och kan efterliknas en ram, denna kan vara synlig. Det sista lagret är margin (marginal) och bestämmer tomrummet runtom elementet.

box-sizing: border-box

Denna CSS-regel ändrar så att istället för att placera padding och border utanför elementets utsatta storlek, så räknas padding och border med i storleken.
Så om vi har ett element med width: 40px, padding: 5px, border: 5px, så får ett element utan border-box (content-box) en faktisk bredd på 60px (40+2×5+2×5).
Men med border-box så är den faktiska bredden samma som width dvs. 40px, i det här sammanhanget blir den faktiska bredden på innehållet bara 20px (40-2×5+2×5) eftersom padding och border växer inåt tar bort storleken på innehållet.

```
<header>
  <div class="header-content">
    <a href="index.html">
      <img src="images/logo.png" class="logo" alt="Gurkans webbshop logga">
    </a>
    <label class="cart-button" for="cart-control">
      <img src="images/cart.png" alt="Varukorg">
    </label>
  </div>
  <div class="header-nav">
    <nav class="header-content">
      <a href="#start">Hem</a>
      <a href="#produkter">Produkter</a>
      <a href="#om-oss">Om oss</a>
    </nav>
  </div>
</header>
```

<header> är ett semantiskt element som visar att innehållet tillhör sidhuvudet, inom HTML är det oftast hemsidans logotyp, navigering och ev. sökfält eller varukorg.

** ** är ett icke-semantiskt element som ofta används för att bygga upp layout på en sida. I min header använder jag 2 div-element för att dela upp den i två rader med vertikalt flytande innehåll i respektive del.

<a> är ett semantiskt element som tyder på en länk, href attributet är sökvägen. I min header omsluter elementet ett **<img>** element som är en bild med hemsidans logotyp. Klickar man på bilden och därmed länken kommer man till webbplatsens index sida.

<nav> är ytterligare ett semantiskt element som beskriver att detta är sidans navigeringsfält.

```
<form class="check-out">
  <div class="form-item">
    <label for="email">E-post:</label>
    <input type="email" name="email" id="email" required>
  </div>
</form>
```

Det här kodutdraget är ett fomulär som används i webbplatsens kassa. **<form>** är grunden till formuläret och innehåller samtliga fält och knappar. **<input>** är ett fält som man kan interagera med, attributet type bestämmer vilken typ av fält det är. ”email” är ett textfält som används för att fylla i e-postadresser. Attributet name gör att formuläret skickar med datan från fältet med namnet ”email” och id ger den en identifierare som **<label>** använder för att länka till fältet. ”required” gör att fältet är obligatoriskt att fylla i.

```
Input {
  width: 100%;
  padding: 5px;
  border: solid 1px #767676;
  border-radius: 2px;
}
```

Den här CSS-regeln används för att styla input-fälten i formuläret. width: 100% gör fältet lika brett som sin förälder och padding: 5px ger den tomrum på insidan åt alla håll. border: solid 1px ger fältet en ram som är 1px bred på alla sidor och border-radius gör hörnen rundade. Eftersom jag använder box-sizing: border-box i min CSS så blir totala bredden på fälten 100%, annars skulle den bli 100%+12px och därav bli större än förälderns bredd.

```
<body>
  <div class="cart"></div>

  <script src="fe26-shop.js"></script>
</body>
```

Varukorgen fungerar med hjälp av dessa element. **<script>** importerar JavaScript filen så att den körs på sidan. JavaScriptet letar i sin tur efter ett (div-?) element med klassen ”cart” och ändrar dess innehåll.

2.

Responsiv design

Handlar om att skapa webbsidor som automatiskt anpassar sin layout, bilder och innehåll efter användarens skärmstorlek — oavsett om det är en mobil, surfplatta eller dator.

Meta **viewport-inställningen** talar om för enheten hur den ska visa sidan, utan den kan mobila webbläsare sätta en standardbredd som gör att sidan inte visas korrekt. width=device-width sätter bredden på sidan till enhetens rapporterade skärmbredd och initial-scale=1.0 sätter den initiala zoomen efter site-load.

Flytande layout & flexibla bilder

Det handlar om att använda sig av en dynamisk layout med element som inte har fasta storlekar och positioner utan som kan anpassa sig efter storleken på viewport. Detta sker ofta med relativa enheter för width och height och moderna CSS-tekniker som Flexbox. Detta gäller även bilder som anpassar sig i storlek.

Media queries & brytpunkter

Media queries är ett sätt att i CSS deklarera regler som bara gäller under vissa villkor. Den vanligaste användningen är att ändra storlek och layout på sidan vid en viss skärmstorlek (viewport bredd) för en responsiv design. Brytpunkter är de bredd-värdena där man ändrar CSS-regler via en media query. Det finns standardiserade brytpunkter som ofta används men de går att sätta till valfritt värde för vad som ser bäst ut med sidans innehåll.

Mobile first

Är ett designsätt där man designar hemsidan mot mobila enheter i första hand och sedan lägger till anpassningar för större skärmar med media queries.

```
.product {
  float: left;
  width: 20%;
  aspect-ratio: 18 / 21;

  @media (width <1120px) {
    width: 25%;
  }

  @media (width <992px) {
    width: 33%;
  }

  @media (width <768px) {
    width: 50%;
    aspect-ratio: 18 / 20.5;
  }
  @media (width <576px) {
    width: 100%;
    aspect-ratio: 18 / 19;
  }
}
```

I min prduktlista använder jag float för att placera produkterna efter varandra och en relativ bredd som gör att en viss mängd produkter får plats på en rad. Bredden och bildförhållandet ändras på elementen baserat på storleken på viewport med hjälp av media queries för att få plats med fler eller färre produkter på samma rad.

I bootstrap har jag löst det med hjälp av deras grid komponent vilket bygger på flexbox. Då satte jag hur många kolumner förälder elementet ska ha för respektive brytpunkt.

I grunden tror jag att min lösning och bootstraps är ganska lika med media queries för brytpunkterna men lösningarna skiljer sig störst i att jag använder float och bootstrap använder flexbox.

3.

En sökmotor som Google arbetar i tre steg: crawling, indexering och ranking där den hittar sidor, sparar och organiserar dem och rangordnar dem baserat på kriterier. Den upptäcker sidor genom att följa länkar från andra webbplatser eller genom så kallade sitemaps. När sökmotorn hittar sidan läser den igenom källkoden och analyserar innehållet. Den tittar på bl.a. title, meta description, semantisk HTML och alt-texter för att avgöra innehållet på sidan och hur den är strukturerad.

Meta-informationen syns generellt inte för användaren på sidan och talar främst om för sökmotorn hur den ska visa sökresultatet. Den semantiska strukturen är det sökmotorn analyserar för att förstå innehållet och avgöra om resultatet är relevant för sökfrasen.

```
<head>
  <meta charset="UTF-8">
  <meta name="description" content="Företaget Gurkans webbshop">
  <meta name="keywords" content="webbshop, e-shop, webshop">
  <meta name="author" content="Gustav Sulingga Wahrberg">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gurkans webbshop</title>
  <link rel="stylesheet" href="style.css">
</head>
```

I en sökmotor visas titeln som rubrik och description står direkt under. Något jag kan förbättra (och gjort sedan jag skrev detta) är att uttöka description med mer beskrivande text.

```
<article id="start">
  <div class="content">
    <h2 class="title">Välkommen till Gurkans Webbshop!</h2>
    <p>Vi är otroligt glada att du har hittat hit. Hos oss kliver du in i ett
    <p>Oavsett om du söker efter någonting som förvandlar ditt hem till en liv
    <section>
      <h3>Handplockade Favoriter för Alla Tillfällen</h3>
      <p>I vårt noggrant sammansatta sortiment möts det överraskande och det s
      <p>Gemensamt för allt du finner hos oss är fokuset på kvalitet, funktion
      <a href="#produkter">Till produktsidan</a>
    </section>
    <section>
      <h3>Varför Handla Hos Oss?</h3>
      <p>Vi strävar alltid efter att kombinera funktion, estetik och hållbarhe
      <p>Vi tror inte på ett enformigt liv – vi tror på en färgstark mix av ny
      <a href="#om-oss">Läs mer om oss här</a>
    </section>
    <p><em>Varmt välkommen att starta din upptäcktsfärd hos oss redan idag – v
  </div>
</article>
```

Jag har satt h2 som huvudrubrik för respektive ”sida” (article) och använder h3 för underrubriker i innehållet. I sökmotorn kan rubriker visas som klickbara länkar eller ersätta titeln om rubriken är mer relevant för sökresultatet.

4.

WCAG är den internationella standarden för webbtillgänglighet och dess fyra principer är:

Möjlig att uppfatta: Innehåll måste presenteras på ett sätt så att användare kan ta del av det, ex. alt-texter för bilder.

Hanterbar: Gränssnitt och navigation ska kunna användas av alla oavsett funktionsnedsättning.

Begriplig: Text och funktioner ska vara lätta att förstå och förutsägbara.

Robust: Innehållet ska fungera på samma sätt på olika enheter.

Webbtillgänglighet täcks av lagstiftning och gäller både den offentliga och privata sektorn sedan juni 2025.

Det ser till att digitala tjänster är tillgängliga och användbara föra samtliga medborgare. Det gynnar speciellt personer med olika funktionsnedsättningar och även äldre men förbättrar användarupplevelsen för alla.

Min webbshop har labels som länkar till formulärfält och alt-texter till bilder. Eftersom min shop använder ett hack för kundvagnen så har jag sett till att den går att komma åt med tangentbordsnavigering. Jag har använt lighthouse för att testa kontraster i text och justerat vid behov.

5.

En CSS-preprocessor är ett verktyg som ger utökad funktionalitet i CSS, ex. variabler, nästning och funktioner. Webbläsare kan inte läsa kod från en preprocessor direkt utan koden måste först översättas och kompileras till vanlig CSS innan webbläsaren kan utnyttja den.

Sass minskar koduppprepning med hjälp av nestning och återanvändbara kodstycken (mixins) så man kan använda samma värden och kod på flera ställen. Bättre kodstruktur med länkade filer och nästning likt HTML struktur, det gör koden lättare att förstå och redigera. Avancerad logik med hjälp av funktioner, loopar, if-statements m.fl.

Jag har använt @extend för att använda Bootstraps CSS regler i element som jag inte kan lägga till klasser på (element från varukorg-scriptet) och vid specifika states. Jag har även använt Bootstraps variabler för färger i min Sass.

Under kursen förstod jag det som att man skulle minska på mängden CSS så mkt som möjligt med Bootstrap och har därför bara skrivit det jag inte kunnat göra direkt i Bootstrap i CSS vilket är mest unika fall och har därför inte använt några fler funktioner.

```
input,
select {
  &:user-invalid {
    @extend .is-invalid;
  }
}
```

Här använder jag Bootstraps styling för form-validering för att markera input- och selectfält som har invalid värden. (Bootstraps egna lösning för validering funkar inte i 5.3)

Modern CSS kan göra nästan alla saker som jag använt i Sass. Det har numera stöd för både variabler, nästning och länkade filer. Det är de tyngre funktionerna som mixins, funktioner, logik som fortfarande bara går att åstadkomma med en preprocessor.

6.

Webbstandarder är regler och riktlinjer för hur HTML och CSS ska utformas. De säkerställer att webbplatser fungerar likvärdigt oavsett vilken webbläsare, enhet eller hjälpmedel användaren använder.

W3C sätter övergripande standarder för webben, bland annat för tillgänglighet (WCAG) och CSS.

WHATWG drivs främst av webbläsarutvecklare och ansvarar för den levande standarden för HTML.

Validering innebär att man kör sin kod genom ett granskningsverktyg för att kontrollera att den följer reglerna och inte har några syntaxfel. Det minskar risken för visningsfel i olika webbläsare, förbättrar tillgängligheten och underlättar sökoptimering.

Värdera: Varför är det viktigt att följa standarderna? Vad kan hända om man inte gör det, för användare, sökmotorer, olika webbläsare och framtida underhåll? Använd egna erfarenheter från valideringen av din shop.

Det är viktigt att följa standarderna för att säkerställa att alla kan använda sidan, att sökmotorer korrekt kan indexera sidan och för att minska risken för buggar.

Om man inte gör det så finns risken att äldre användare eller användare med funktionsnedsättningar inte kan använda sig av sidan. Sökmotorer kommer inte visa sidan som sökresultat till relevanta termer om den inte kan tyda sidan korrekt. Olika webbläsare och enheter kan få svårt att visa sidan korrekt om den inte anpassas efter olika webbläsare och till mobila skärmstorlekar. Till sist förenklar en semantiskt korrekt struktur för framtida revisioner och uppdateringar till sidan för att den blir lättare att förstå hur den är uppbyggd.

I min shop hade jag strö-taggar som skulle kunnat påverka en webbläsare som inte har normalization/error correction inbyggt. Jag saknade en del rubriker som är extra viktigt för användare med skärmläsare att förstå sidans struktur. När jag testade sidan på en mobil enhet upptäckte jag att produktlistan var för inzoomad och att en kolumn inte var optimalt för användarupplevelsen så jag ändrade i mina media queries att sidan skulle visa två kolumner även på de minsta storlekarna.

7.

Jag föredrog att skriva sidan utan ett ramverk, i alla fall när vi använde Bootsrap på det sättet vi gjorde nu med så lite egen CSS som möjligt, eftersom det ger mer finstilt kontroll av storlekar och så vidare. Den största fördelen med Bootstrap är de förbyggda komponenterna med färdig styling som enkelt går att lägga in, som i mitt fall offcanvas som jag behövde utnyttja ett hack för att få till i vanilla utan JavaScript och den färdiga stylingen av formulär och accordion komponenten gjorde stor skillnad. En stor nackdel är speciellt nu när vi är nya till Bootstrap så måste man söka upp alla namn på klasser utan någon hjälp av autocomplete i ex. VS Code. Dokumentationen är okej för bootstrap men kan inte jämföras med mängden som finns för standard CSS i ex. MDN och W3Schools.

I Bootstrap fick jag mer HTML och mindre CSS jämfört med vanilla versionen, speciellt fick Bootrstrap versionen fler klasser på element.

Jag skulle välja att använda ramverk i mindre, kortare projekt där jag snabbt vill bygga upp en fungerande hemsida som ser snygg och konsekvent ut med färdiga komponenter osv.
I ett längre projekt kan det variera men för mer kontroll och en unikare look så skulle jag nog hellre välja att utveckla sidan utan ramverk.

Jag använda AI för att generera text och bilder till sidans innehåll.
För att snabbt duplicera kod jag hade ändrat till andra delar av koden där liknande kod förekom, ex. om jag ändrade några element i en av produktsidorna så kopierade jag den ändringen till de andra produktsidorna fast med respektive produkts unika innehåll.
Jag använde även AI för att snabbt hitta lösningar till mina problem, för det mesta var det mer pålitligt att gå in i dokumentation eller forum.